

Full Stack Developer Assessment (Advanced Level)

Section 1: System Design & Architecture

1. Design a scalable multi-tenant SaaS platform that supports thousands of organizations with isolated data, role-based access, and real-time updates. Explain your architecture decisions.
 2. How would you design a system similar to a project management tool (like task tracking, comments, attachments, activity logs)? Focus on scalability, data consistency, and performance.
 3. Design a real-time notification system that supports in-app notifications, email, and push notifications for millions of users.
 4. How would you design a file upload and processing system that handles large files (videos/images), including storage, processing, and delivery?
 5. Design an offline-first application that syncs data when the connection is restored. How would you handle conflicts?
-

Section 2: Backend Engineering (Node.js / Java / Python)

6. Design and implement a REST API for a resource that includes:
 - Authentication
 - Authorization (role-based)
 - Pagination
 - Filtering
 - Rate limiting
 7. Explain how you would structure a large-scale backend application for maintainability and scalability.
 8. How would you handle concurrency and race conditions in a high-traffic backend system?
 9. Design a background job processing system (queues, retries, failure handling).
 10. How would you implement API versioning in a production system without breaking existing clients?
-

Section 3: API & Integration

11. Design a RESTful API for a system where users can create, update, delete, and share resources with others.
 12. How would you design an API gateway in a microservices architecture?
 13. Explain how you would handle third-party API failures in a production system.
 14. How would you secure APIs against common vulnerabilities?
 15. Design a webhook system that external services can subscribe to.
-

Section 4: Database Design (SQL + NoSQL)

16. Design a database schema for a system that includes users, roles, permissions, and activity logs.
 17. How would you design a system that requires high read performance and moderate writes?
 18. When would you choose SQL over NoSQL and vice versa in a production system?
 19. Design a data model for storing hierarchical data (e.g., categories, comments).
 20. How would you handle database migrations in a live production environment?
-

Section 5: Frontend Engineering (React / Modern JS)

21. Design a complex dashboard UI that handles large datasets efficiently.
 22. How would you manage global state in a large frontend application?
 23. Explain how you would optimize performance in a React application.
 24. How would you handle form validation and dynamic forms at scale?
 25. Design a component architecture for a reusable UI system.
-

Section 6: Full Stack Integration

26. How would you structure communication between frontend and backend in a large-scale application?
27. Design a complete flow for user authentication (including token management and session handling).
28. How would you implement real-time updates (e.g., live chat or live dashboard)?
29. How would you handle errors across the full stack (frontend + backend)?
30. Design a logging and monitoring system for a full-stack application.



Section 7: Performance & Scalability

31. How would you scale a system from 1,000 to 1 million users?
 32. Explain how caching works and where you would use it.
 33. How would you optimize a slow API endpoint?
 34. Design a system to handle traffic spikes (e.g., flash sales).
 35. How would you reduce latency in a global application?
-



Section 8: Security

36. How would you implement authentication and authorization securely?
 37. Explain how you would prevent common attacks (XSS, CSRF, SQL Injection).
 38. How would you secure sensitive user data?
 39. Design a secure password reset system.
 40. How would you manage secrets in a production environment?
-



Section 9: Problem Solving & Real Scenarios

41. A system is experiencing random downtime under load. How would you debug it?
 42. Users report data inconsistency across devices. How would you investigate?
 43. A third-party integration is slowing down your system. What would you do?
 44. Your database queries are becoming slow as data grows. How would you fix this?
 45. A deployment caused critical bugs in production. What steps would you take?
-



Section 10: Practical Task (Mandatory)

46. Build a full-stack application that includes:
 - User authentication
 - CRUD operations
 - Role-based access
 - API integration
 - Responsive frontend
 - Proper error handling
47. Implement a REST API with proper structure, validation, and security.

48. Design the database schema for your application.
49. Ensure the application is scalable and production-ready.
50. Provide deployment strategy and environment setup.